Project Report:

Smart Emergency Hospital Locator

Submitted by: Naman Singh (25BAI11153)

Course: CSE — Artificial Intelligence & Machine Learning

1. Introduction

In emergency medical situations, time is the most critical factor. Finding the nearest hospital is not just about straight-line distance but about navigable road routes. This project aims to solve the "Nearest Facility" problem by identifying hospitals within a specific radius and determining the most efficient path using geographic data and search logic.

2. Problem Statement

When a user is in an unfamiliar location or an emergency arises, they need an automated way to:

- Identify their current geographic coordinates.
- Locate all medical facilities (hospitals) within a 5km radius.
- Calculate the actual road distance to these facilities.
- Determine the optimal (nearest) hospital to minimize travel time.

3. Implementation Logic & Algorithm

The core logic of this project is based on **Breadth-First Search (BFS)** principles applied to a geographic graph.

3.1 Breadth-First Search (BFS) in Navigation

In traditional AI, BFS is used to explore nodes in a graph layer by layer. In this project:

- **Root Node**: The user's current location.
- **Neighbor Nodes**: Nearby hospital coordinates retrieved via the OpenStreetMap (OSM) Overpass API.
- **Search Strategy**: The algorithm "explores" the surrounding 5km radius (the first "layer" of the search space) to find all valid hospital nodes before evaluating the cost (distance) to each.

3.2 Technical Workflow

1. **Geocoding**: Uses the geopy library to convert a text-based address into Latitude and Longitude coordinates.
2. **Spatial Querying**: Connects to the **Overpass API** to find nodes tagged as "amenity=hospital" within a 5,000-meter circular buffer.
3. **Distance Calculation**: Utilizes the **OSRM (Open Source Routing Machine) API** to fetch real-world driving distances, which accounts for road networks rather than simple "as-the-crow-flies" distance.
4. **Optimization**: Evaluates the distances and identifies the minimum value to recommend the best facility.

4. System Architecture

The system is built using Python and relies on three major external integrations. Python 3.x is used for the core logic and API handling. For geocoding, Nominatim via the Geopy library converts user input to GPS coordinates. The primary data source is OpenStreetMap (OSM), which provides real-time hospital location data. Finally, the OSRM API serves as the routing engine to calculate driving distances between nodes.

5. Key Code Functions

- **hsl()**: Handles user input and error checking for location accuracy.
- **hospital_loc(lat, lon)**: Queries the Overpass API and parses JSON data to extract hospital names and coordinates.
- **hkv(clat, clon, hlat, hlon)**: Interfaces with OSRM to return the driving distance in kilometers.
- **main()**: Orchestrates the flow and performs the final comparison to find the minimum distance.

6. Results & Conclusion

The application successfully identifies local hospitals and ranks them by proximity. By leveraging BFS-style spatial searching and real-time routing APIs, the project provides a functional tool for emergency response.

Future Enhancements:

- Integrating real-time traffic data into the BFS weight calculation.
- Developing a GUI for faster user interaction.